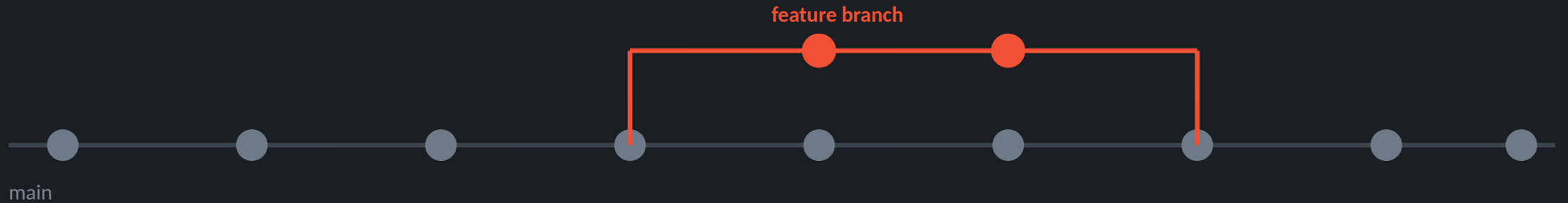


HANDS-ON WORKSHOP

Git & GitHub

From your first commit to collaborating on real projects

~90 minutes · No prior experience needed · Bring a laptop



Six things you will walk out knowing



Why version control

Track every change, undo mistakes, and stop naming files `final_v3_REAL`.



Git vs GitHub

Two different things that people constantly mix up.



The core workflow

Working directory to staging to commit to remote.



Branching & merging

Build features without ever breaking main.



Collaborating

Push, pull, fetch, and pull requests on GitHub.



Undo & recovery

restore, revert, reset, and the reflow safety net.

Git is a time machine for your project

Git is a version control system. It records snapshots of your files over time, so you can see what changed, when, and by whom — and return to any earlier version of the project.

Git runs locally on your computer, so it works with no internet at all.

Monday

Tuesday

Wednesday

Thursday



git restore · git revert · git reset — take you back to any earlier snapshot

GitHub is where your Git projects live online



GitHub

An online platform that hosts Git repositories.
Think of it as cloud storage for Git projects, with
a full set of collaboration tools bolted on.

Git runs on your machine.
GitHub runs in the cloud.



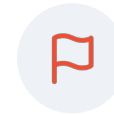
Store repositories online



Collaborate with a team



Branches and pull
requests



Track issues and bugs



Review other people's
code



Share work and build a
portfolio

Git vs GitHub: not the same thing



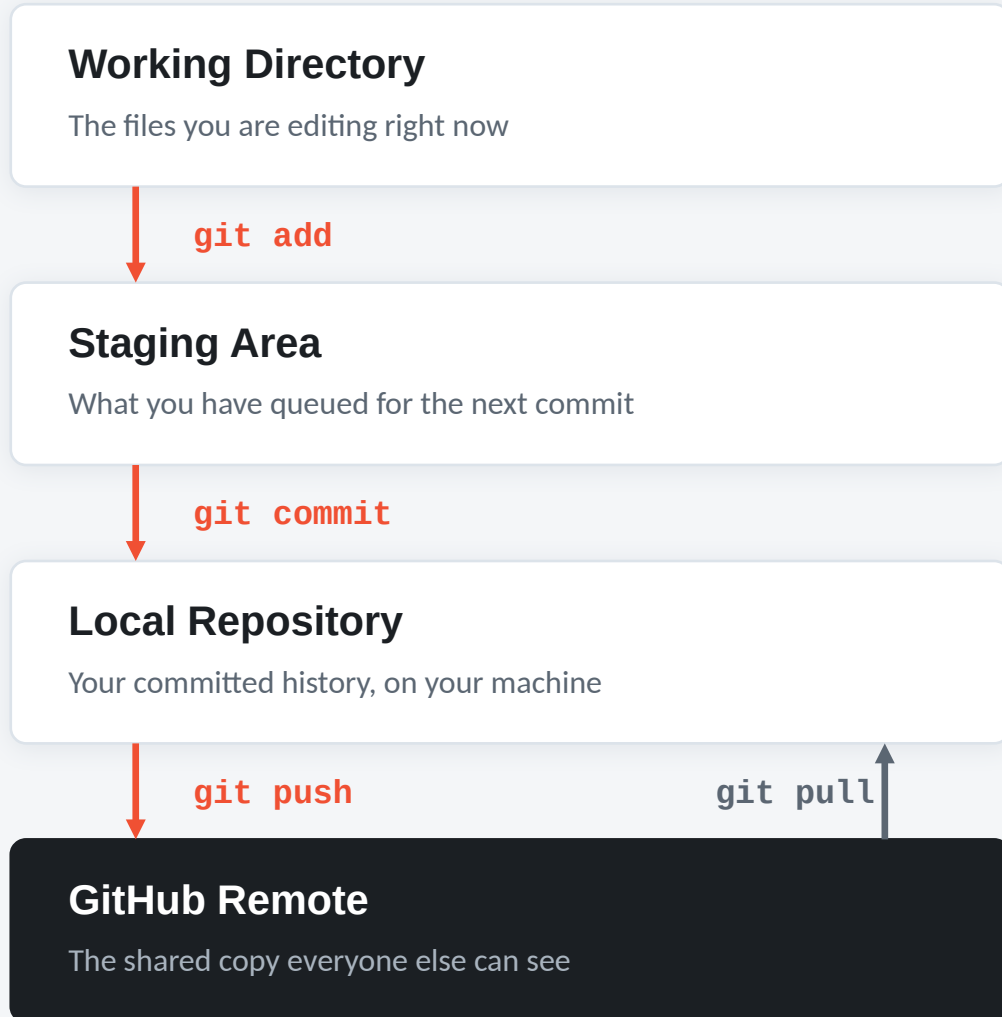
- Software you install
- Runs on your computer
- Tracks your project history
- Works fully offline
- Created by Linus Torvalds



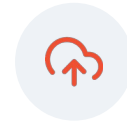
- An online platform
- Runs in the cloud
- Hosts Git repositories
- Needs internet for remote work
- Owned by Microsoft

Git manages your code's history. GitHub stores that history online and lets other people work on it with you.

The one workflow to memorise

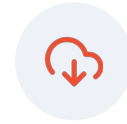


The three remote verbs



git push

Uploads your local commits to GitHub.



git pull

Downloads remote changes AND merges them into your branch.



git fetch

Downloads remote changes but merges nothing. Look before you leap.

Commits are local until you push them.

Set Git up once on each machine

```
git --version
```

Check that Git is installed, and which version

```
git config --global user.name "Name"
```

The name attached to every commit you make

```
git config --global user.email "email"
```

The email attached to every commit

```
git config --list
```

Review everything Git currently knows about you

```
git help <command>
```

Built-in documentation for any command

Your name and email are baked into every commit, so set them before your first one.

Every project starts one of two ways



Start something new

```
git init
```

Turns the current folder into a Git repository. Nothing leaves your machine yet.



Join something existing

```
git clone <url>
```

Copies a remote repository — every file and its whole history — onto your computer.

Connecting a local repo to GitHub

```
git remote add origin <url>
```

Link your local repo to a repository on GitHub

```
git remote -v
```

Show which remotes this repo is connected to

```
git remote set-url origin <url>
```

Point the repo at a different remote URL

The loop you will repeat all day

01

Check

```
git status
```

See what changed and what is staged

02

Stage

```
git add .
```

Choose what goes into the next commit

03

Commit

```
git commit -m "..."
```

Save a snapshot with a message

04

Share

```
git push
```

Send your commits up to GitHub

Look before you commit

```
git diff
```

Unstaged changes

```
git diff --staged
```

What is about to be committed

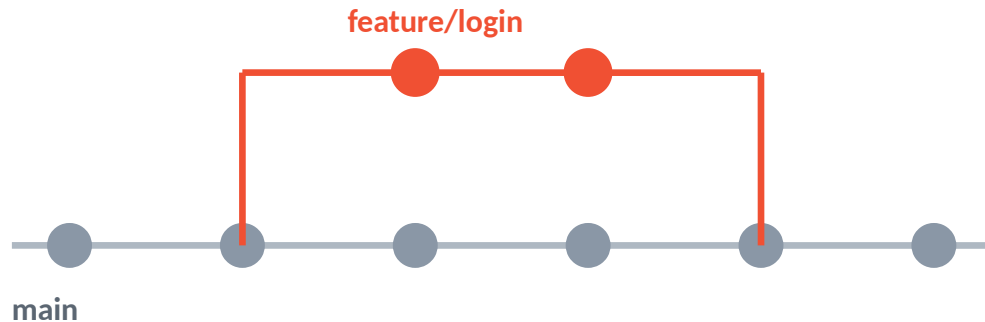
```
git log --oneline
```

Compact commit history

Commit early and often. A good message finishes the sentence: "This commit will..."

Work without breaking main

A branch is a separate line of work. You experiment freely, and main stays safe until you merge.



Branch off, build, then merge back in.

Branch commands

<code>git branch</code>	List local branches
<code>git switch -c <name></code>	Create and switch (modern)
<code>git switch <name></code>	Switch to a branch
<code>git checkout -b <name></code>	Older way to create + switch
<code>git branch -d <name></code>	Delete a merged branch
<code>git branch -m <name></code>	Rename the current branch

Bringing branches back together

01

```
git switch main
```

Move to the branch that should receive the work.

02

```
git merge feature
```

Pull the feature branch into the current branch.

03

```
git branch -d feature
```

Tidy up once the work is safely merged.



When Git can't decide: merge conflicts

- Git marks the clashing section inside the file with <<<<<< and >>>>>> markers.
- Open the file, keep the version you want, and delete the marker lines.
- Then git add <file> and git commit to finish the merge.
- Want out? git merge --abort returns everything to how it was.

Push, pull, fetch — know the difference



Push

```
git push -u origin main
```

Uploads your local commits to GitHub. The `-u` sets upstream tracking, so after the first time `git push` alone is enough.



Pull

```
git pull
```

Downloads changes from the remote AND merges them into your current branch. Do this before you start working.



Fetch

```
git fetch
```

Downloads changes but merges nothing. Lets you inspect what other people did before it touches your work.

```
git push origin main
```

Push a specific branch to origin

```
git fetch --all
```

Fetch from every configured remote

```
git push origin --delete <branch>
```

Delete a branch on the remote

Undo, safely

```
git restore <file>
```

Throw away unstaged edits in that file

```
git restore --staged <file>
```

Unstage a file but keep the edits

```
git revert <commit>
```

New commit that undoes an old one — safe on shared branches

```
git reset --soft <commit>
```

Move HEAD, keep the changes staged

```
git reset --mixed <commit>
```

Move HEAD and unstage the changes

```
git reset --hard <commit>
```

Move HEAD and discard the changes



Careful

git reset --hard can permanently discard uncommitted work. There is no undo for work Git never recorded.

On anything you have already pushed, reach for git revert instead.



git reflog

shows every move HEAD has made — including commits you thought you destroyed. This is how people recover "lost" work.

Two tools that make life easier



Stash

Unfinished work, but you need to switch branches right now.

git stash	Shelve uncommitted changes
------------------	----------------------------

git stash push -m "msg"	Shelve with a description
--------------------------------	---------------------------

git stash list	See everything you shelved
-----------------------	----------------------------

git stash pop	Restore the latest and remove it
----------------------	----------------------------------

git stash apply	Restore but keep the stash
------------------------	----------------------------



Tags

Mark a specific commit as a version or release.

git tag	List existing tags
----------------	--------------------

git tag v1.0	Create a lightweight tag
---------------------	--------------------------

git tag -a v1.0 -m "msg"	Create an annotated tag
---------------------------------	-------------------------

git push origin --tags	Push tags to GitHub
-------------------------------	---------------------

git tag -d v1.0	Delete a local tag
------------------------	--------------------

Master these fifteen first

01

`git --version`

02

`git config`

03

`git init`

04

`git clone`

05

`git status`

06

`git add`

07

`git commit`

08

`git log`

09

`git branch`

10

`git switch`

11

`git merge`

12

`git remote`

13

`git push`

14

`git pull`

15

`git fetch`



Everything else — rebase, cherry-pick, bisect, blame, reflog — can wait until these fifteen feel automatic.

Your turn: one repo, end to end

1

Make a folder, open a terminal in it, and run `git init`

2

Create `README.md`, then run `git status` to see it untracked

3

`git add README.md`, then `git commit -m "Initial commit"`

4

Create an empty repository on GitHub and copy its URL

5

`git remote add origin <url>` to link the two together

6

`git push -u origin main` to publish your history

7

`git switch -c feature/about`, add a file, and commit it

8

Push the branch, then open a Pull Request on GitHub

THE ONE THING TO REMEMBER

Git manages your code's history. GitHub stores and shares it.



Commit small, commit often

A commit you can describe in one line is a commit you can undo.



Branch for every feature

main should always be something you could ship.



Push before you log off

Unpushed work exists in exactly one place: your laptop.

Stuck on anything? `git help <command>` · docs.github.com